

"Treasure Hunt Adventure"

☐ SAED RADI [REDACTED]

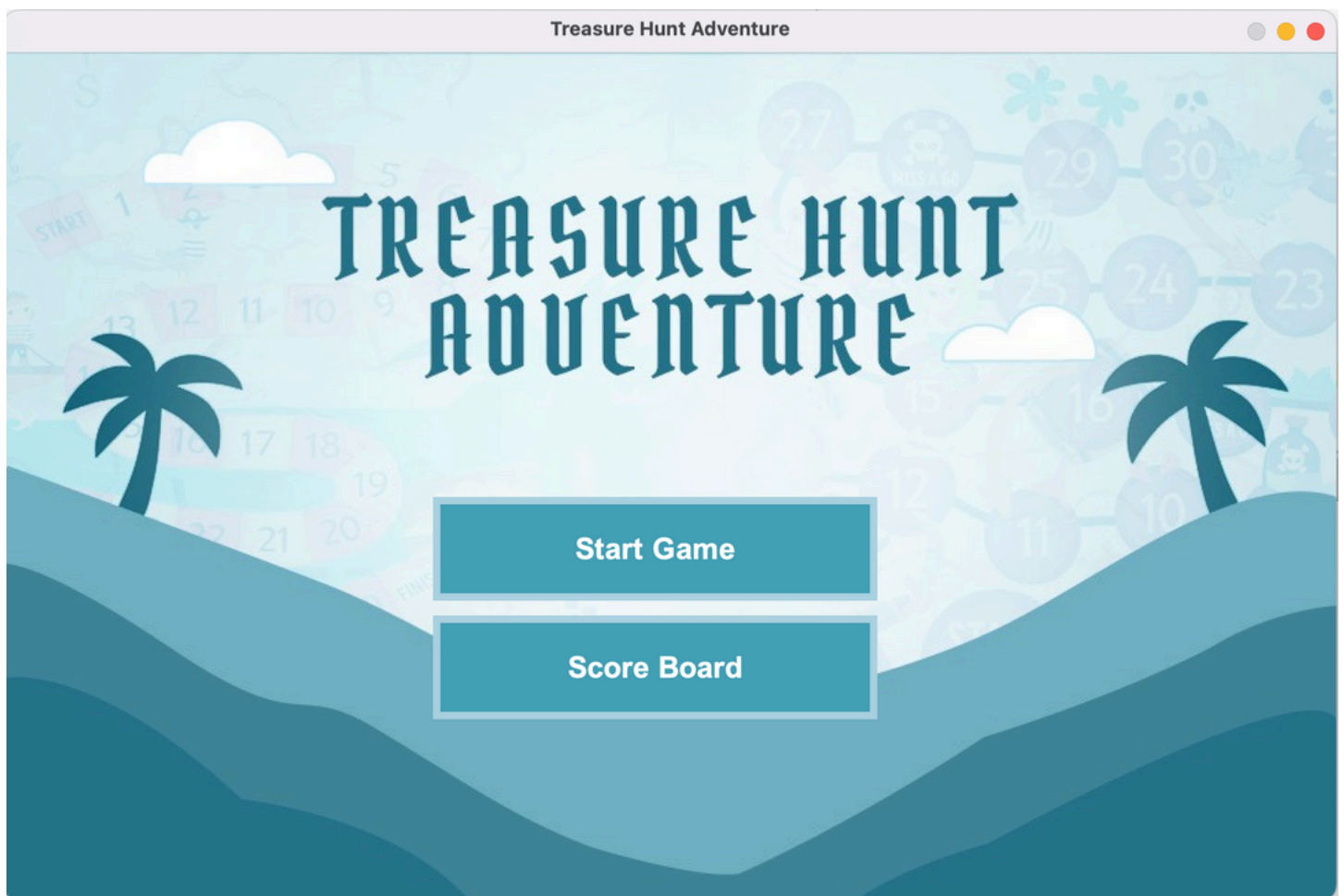
☐ ABDULRAHMAN ZEINEDDIN [REDACTED]

This project, developed as part of the *Data Structures* course requirements, aims to design and implement a simulation game called "*Treasure Hunt Adventure*" using the *Java* programming language. The game relies on key data structures such as the *Linked List* and *Binary Search Tree (BST)*.

The game is designed to provide an interactive environment for the user. The player starts by entering a username and then goes through two levels of challenges on maps made up of connected cells representing game spots. Each cell may contain a specific event such as a treasure, trap, or movement instructions. The *linked list* structure is used to organize and link the cells sequentially, while the *binary search tree* is used to store and analyze the player's scores after each level.

- Menu

When the game is launched, the user is greeted with the main menu interface, which is designed to be visually appealing and simple, as shown in the image below:

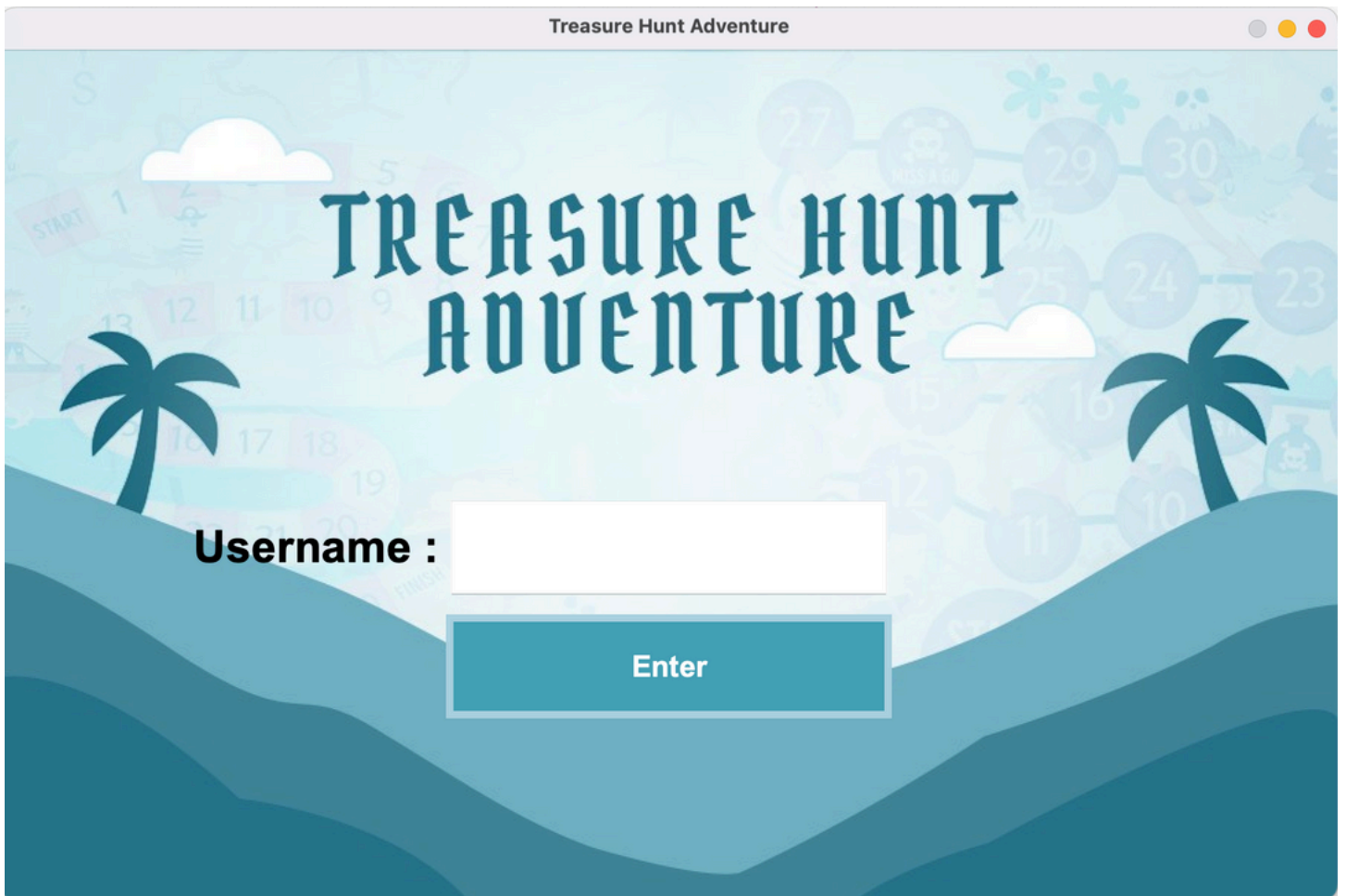


Interface Components:

- The background image was custom-designed using [Adobe Photoshop](#), featuring the game title "**TREASURE HUNT ADVENTURE**" and decorative elements inspired by a tropical treasure island.
- This background was then integrated into the Java interface using code from the [Java Swing GUI library](#).
- Two main buttons were added programmatically:
 1. **Start Game**: When clicked, it navigates to a frame that prompts the user to enter their username. After entering the name, the first level of the game begins.
 2. **Score Board**: When clicked, it navigates to a frame that also prompts for a username, but instead of starting the game, it displays the user's past scores by reading data from a [file](#) and constructing a [Binary Search Tree \(BST\)](#).

• **Username**

This interface appears immediately after the user clicks either the "**Start Game**" or the "**Score Board button**" from the [menu](#).



Interface Components:

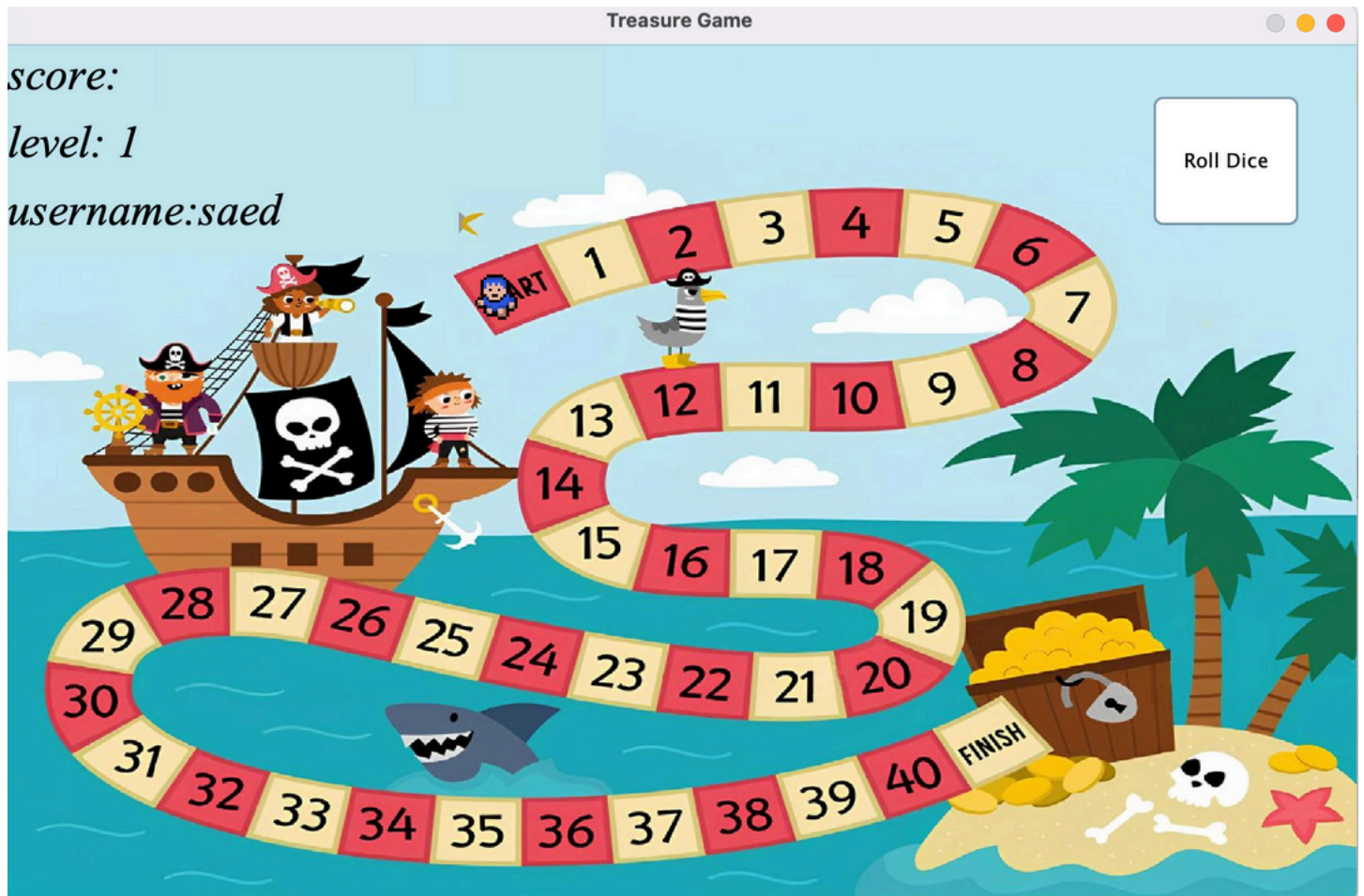
- The background remains the same as in the [menu](#), designed using *Adobe Photoshop*, and reused here to maintain design consistency.
- The frame contains:
 - A *text field* that allows the user to input their username.
 - An *"Enter" button* that responds according to the context.

Functional Behavior:

- If the user navigated via the *"Start Game" button*, pressing *"Enter"* will launch [Level 1](#) of the game.
- If the user reached via the *"Score Board" button*, pressing *"Enter"* will lead to the [scoreboard](#), where the user's scores are retrieved and displayed using a *Binary Search Tree*.
- This dynamic behavior is handled through conditional logic based on the user's navigation path.

- [Level 1](#)

For the implementation of **Level 1**, we used a custom *JPanel* rather than a *JFrame* as the main drawing surface. This decision was made specifically to allow custom rendering of images using the *paintComponent(Graphics g)* method. By using *JPanel*, we were able to achieve higher visual quality and more precise control over image placement, scaling, and layering. This approach also ensured that all graphical elements, such as the background and the player icon, were rendered smoothly and in the correct positions.



Graphical Background:

The background image used in **Level 1** was initially provided by the instructor. Minor visual modifications were applied using a graphic design tool to better fit the context of the level. The image includes thematic elements such as a pirate ship, a path, and a treasure island, all reinforcing the adventurous atmosphere of the game.

Player Information Display:

In the top-left corner of the screen, key player information is displayed:

- **Username**
- **Current level**
- **score**

This section was implemented using *JLabel* components from the *Java Swing library*. Font styles, colors, and layout positions were manually configured in the code for a clear and organized user interface.

Game Board Design with Two Linked Lists:

The board for Level 1 is structured using two separate *linked lists*, each serving a specific purpose within the logic of the game.

The first list, *pathPoints*, stores the coordinates (x, y) of each tile along the visual path. These points are used purely for rendering the player's position on the background image.

To determine the exact positions on the background image, we used a *MouseListener* temporarily during development. It allowed us to click on the image and print out precise (x, y) values to be added into the list. This made it easier to map the player's path visually and accurately.

The second list, *board*, represents the actual logical path the player progresses through. Each node contains the same coordinate data from *pathPoints* along with a randomly assigned type:

- *type = 1: Treasure (+10 points)*
- *type = 2: Trap (-5 points)*
- *type = 0: Empty (no change)*

This list is traversed using a special pointer *ptemp*, which advances based on the result of a dice roll. Separating visual coordinates from game logic provides flexibility and clarity in controlling both rendering and gameplay behavior.

Dice Rolling Mechanism:

A *"Roll Dice" button* is placed in the top-right section of the interface. When clicked, it initiates an animated dice rolling effect. This is achieved by rapidly switching between six preloaded *ImageIcon* objects representing dice faces, using a *Timer* set to a short delay (approximately 150 milliseconds) for 15 iterations.

Once the animation completes, a random number between 1 and 6 is selected as the final result of the roll, and the button image is updated accordingly. The game then proceeds to move the player on the board.

Player Movement:

Based on the dice value, the player is moved forward through the board list. This movement is animated using another *Timer*, updating the player's position one step at a time. Each step advances the *ptemp pointer* and triggers a *repaint* of the player icon at the new coordinates.

When the movement ends, the *scoreupdet method* is called. This method checks the type of the current tile and updates the score accordingly:

All tile icons such as *chest, trap, empty* were custom-designed using *Photoshop* to ensure a consistent and thematic visual style throughout the game.

- *Treasure adds 10 points*
- *Trap subtracts 5 points*
- *Empty tile leaves the score unchanged*

A relevant icon (e.g., chest or trap) is also displayed on screen to reflect the tile content.

End of Level:

When the player reaches the final node in the *linked list*, the level ends automatically. At this point:

- The player's *username*, *level* number, and final *score* are saved to an external text *file* via the *Files*.
- A dialog box appears, asking whether the player wants to continue to the *level 2* or return to the *menu*.
 - If the player selects "*Yes*", the *level 2* is loaded.
 - If "*No*" is selected, the *menu* interface is displayed again.

• Level 2

Similar to *Level 1*, *Level 2* was implemented using a *JPanel* rather than a *JFrame*. This choice allowed us to override the *paintComponent(Graphics g)* method to achieve higher control over rendering, enabling smoother image drawing and better visual quality across the board.



Graphical Background:

The background image for this level was custom-designed using [Adobe Photoshop](#), with a theme that reflects a treasure map. The tile positions and path were based on the same visual map provided by the instructor, ensuring alignment with the expected structure.

Dual Linked List Structure:

Level 2 also utilizes two distinct linked lists:

- **pathPoints**: A doubly linked list that stores only the (x, y) coordinates of each tile. These points are used solely for rendering the player's position accurately on the background image.
- **board**: A doubly linked list that represents the logical structure of the game. Each node stores a coordinate, a type, and a value. The type determines the nature of the tile:
 - **0: Empty**
 - **1: Chest (+10 points)**
 - **2: Trap (-5 points)**
 - **3: Move forward (additional steps)**
 - **4: Move backward (steps in reverse)**

Each tile's type and value are randomly assigned at the start of the game, creating a new experience every time the level is launched.

All tile icons such as *chest, trap, empty, forward, and backward* were custom-designed using *Photoshop* to ensure a consistent and thematic visual style throughout the game.

Dice Roll and Movement Logic:

Upon clicking the *"Roll Dice" button*:

- An animated dice roll is displayed using a sequence of images.
- After the animation, a random number between 1 and 6 is generated.
- The player is moved forward through the board list based on the dice value.

Once the movement is complete:

- The current tile is evaluated, and its effect is applied via the *scoreupdet method*.
- If the tile contains a movement value (type = 3 or 4), a delay is introduced before performing the additional movement, and a special icon and message are shown to the player.

Visual Feedback and Score Display:

Level 2 includes three dedicated display areas at the top:

- One shows the current tile's type (e.g., chest, trap).
- Another shows the movement type (forward or backward).
- A third displays the number of additional steps to be taken (if applicable).

These are implemented using *JLabel* components with updated icons and text during gameplay, offering real-time feedback to the player.

End of Level:

When the player reaches the final tile:

- Their *username, level*, and final *score* are saved to a text *file* using the *Files*.
- A message box appears to thank the player for completing the game.

Design Revision Note:

It is important to note that *Level 2* was initially implemented using fixed tile types for each node on the board. However, after reviewing the project with our instructor, it was clarified that the tile types were expected to be assigned randomly with each run. To comply with this requirement, we restructured the entire level from scratch.

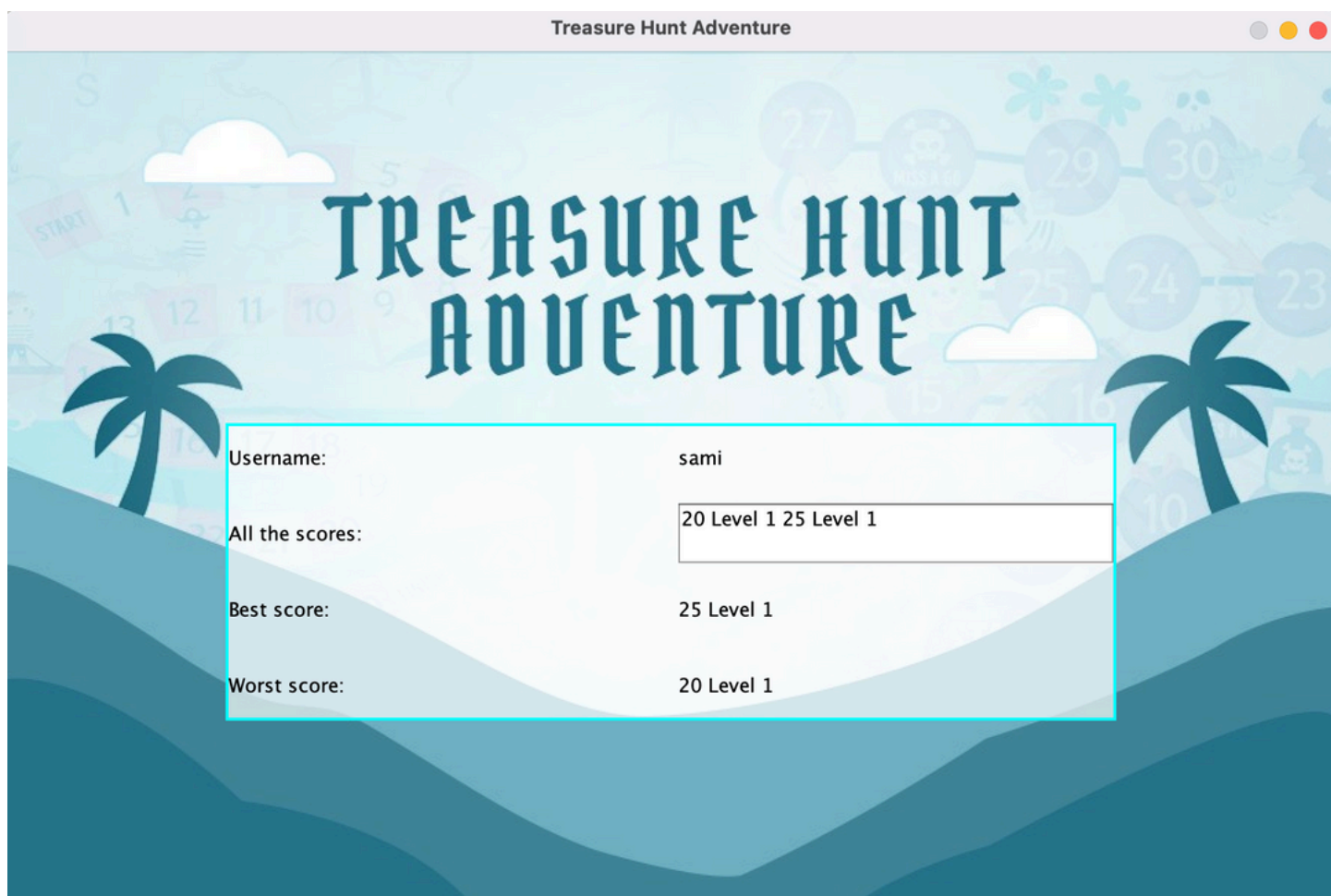
This included:

- Designing a new visual map tailored for dynamic behavior.
- Modifying the board logic to support random type and value generation for each tile.
- Updating the movement and scoring system to adapt to the new randomized structure.

This revision significantly enhanced the reusability and replayability of the level, as each game session now presents a unique path and challenge for the player.

- **Scoreboard**

The scoreboard screen was implemented using a *JPanel* instead of a *JFrame* to allow custom image rendering through *paintComponent*, ensuring higher image quality. The background image, designed using *Photoshop*, spans the entire *panel* and adds a thematic feel to the interface.



The infoPanel includes the following fields:

- *Username*
- *All scores (sorted)*
- *Best score*
- *Worst score*

To improve readability when displaying multiple scores, the *"All scores"* section was implemented using a *JTextArea* embedded inside a *JScrollPane*, instead of a simple *JLabel*.

This allows for text wrapping and vertical scrolling when the number of scores exceeds the available space.

File Handling and Data Extraction:

The game maintains a persistent record of scores in a plain text *file* named *score.txt*. Each line in the file represents a single game result, formatted as:

username, level, score

When the user inputs their name in the **scoreboard** interface:

- The system opens the *file* and reads it line by line using a *Scanner*.
- Each line is split into its components (*username, level, score*).
- Only the lines that match the entered *username* (case-insensitive) are processed.
- Each matching record is inserted into a *Binary Search Tree (BST)* using the score as the key.

This approach allows multiple scores from the same user (across different levels or attempts) to be handled efficiently and displayed in a meaningful way.

Binary Search Tree (BST) Implementation:

A custom *BinarySearchTree* class was developed to manage and analyze the user's scores. Each node in the tree (of type *Tnode*) contains:

- *score (used as the key for sorting)*
- *username*
- *level*
- *left and right child nodes*

When a score is added via the *traversal method*, it is inserted into the tree recursively based on its numeric value. This allows the tree to automatically organize all the user's scores in a sorted structure.

The scoreboard then uses the *BST* to:

- *Retrieve all scores* using an *in-order traversal (inOrder())*, which returns the scores sorted from lowest to highest.
- *Find the highest score* by traversing to the *rightmost node (getMax())*.
- *Find the lowest score* by traversing to the *leftmost node (getMin())*.

These values are displayed in the *GUI* under the respective labels.

Summary:

This integrated approach using a *Binary Search Tree and file handling* provides a scalable and efficient way to manage multiple user scores. The use of sorted traversal enables clear presentation of user progress, while the dynamic *GUI* updates ensure a user-friendly

experience. Additionally, the modular design allows for easy extension, such as filtering by level, or tracking scores over time.

• Challenges

During the development of the project, we encountered several technical and design-related challenges, including:

- *Generating the game map randomly* while ensuring a balanced distribution of tile types (e.g., chest, trap, forward, backward) to maintain gameplay variety.
- *Handling file input and output* in a flexible and efficient way that supports storing and retrieving user data across multiple sessions.
- *Designing and implementing the Binary Search Tree (BST)* to insert and retrieve scores correctly, as well as extract maximum and minimum values for display.
- *Maintaining a user-friendly interface*, despite the complexity of the game logic and data structures working in the background.

• Conclusion

The *Treasure Hunt project* helped us apply *data structure* concepts in a practical and fun way.

We used *linked lists* to create and control the game map, and a *binary search tree (BST)* to organize and display player scores.

All project requirements were successfully completed, and we fully tested the game to make sure everything works as expected.